

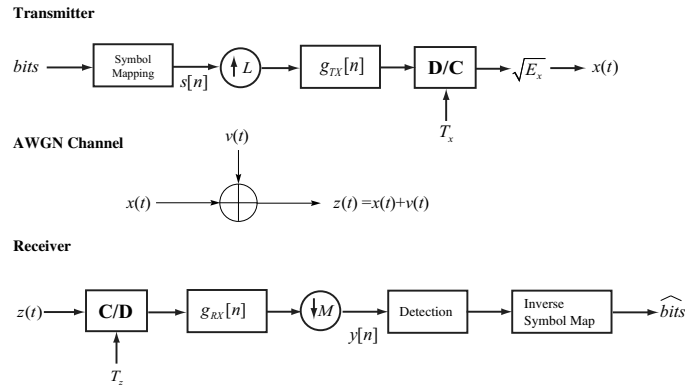
## Lab 2: Baseband QAM Modem

### Part 1: Modulation and Detection

#### Summary

In Lab 2, you will build a baseband digital modem that can use binary phase-shift keying (BPSK) or quadrature phase-shift keying (QPSK) modulation. The complexity of the transmitter and the receiver will grow in subsequent labs as more kinds of receiver impairments are considered. The main concepts in this lab include constellation mapping, oversampling, pulse shaping, matched filtering, maximum-likelihood detection, and probability of error calculation.

The system considered in this lab is illustrated in Figure 1. This modem transmits symbols and applies a detection algorithm assuming only an additive white Gaussian noise (AWGN) channel. The transmitter maps bits to elements of a symbol constellation. The sequence of symbols is upsampled then filtered in discrete-time by a transmit pulse shape. The filtered sequence



**Figure 1:** The system under consideration in this lab. The transmitter uses digital pulse-shaping and upsampling to create the transmit waveform. The baseband signal encounters additive white Gaussian noise during transmission. The receiver uses digital pulse-shaping and downsampling, followed by detection, to find a good guess of the transmitted symbols.

is then passed to the discrete-to-continuous converter. The receiver samples the received signal and filters it with the receiver pulse shaping filter. The filtered symbols are passed to a detection block which determines the most likely transmitted symbol for that observation. The detected symbols are passed to an inverse symbol mapping block to produce a good guess of the transmitted bits.

This lab is broken up into two parts. In this first part, you will build the modulation and decoding blocks of the modem in LabVIEW. After verifying the correctness of your design using a simulator, you will implement your code in lab using RF Hardware to see how your code works over a real wireless link. The goal of this part of Lab 2 is to introduce the basic concepts of modulation and detection. In the second part, you will implement a pulse-shaping filter (on the transmitter side) and the matched filter (on the receiver side).

You will be required to turn in two VIs (*modulate.vi* and *demodulate.vi*), plot the BER curves for QPSK and BPSK, and answer the questions in Section 2, as a part of the pre-lab submission. The lab report for this part of the lab will need to be submitted with the lab report in Part 2 of Lab 2 (i.e., there is no lab report required for this part of the lab).

## 1 Background

This section provides some review of the concepts of digital modulation and maximum likelihood detection. The emphasis is on BPSK and QPSK modulation, with maximum likelihood detection assuming AWGN.

### 1.1 Modulation

Digital modulation consists of mapping a sequence of bits to a (finite) set of analog waveforms which are suitable for transmission over a particular channel. There are many different kinds modulation. This and subsequent labs focus on what is known as complex pulse amplitude modulation. With this modulation technique, symbols are modulated onto pulse-shapes.

The source for digital modulation is a sequence of bits  $\{b[n]\}$ . The bit sequence is passed to the physical layer of a radio by higher layers.

The sequence of bits is processed by the symbol mapping block to produce a sequence of symbols  $\{s[n]\}$ . Essentially, each value  $s[n]$  is a complex number that comes from a finite set of symbols called the *constellation* and written as

$$\mathcal{C} = \{s_0, \dots, s_{M-2}, s_{M-1}\}.$$

The entries of the constellation are different (possibly complex) values. The size of the constellation, or cardinality, is denoted  $|\mathcal{C}| = M$  where  $M$  is the number of symbols in the constellation. For practical implementation  $M = 2^b$  where  $b$  is the number of bits per symbol so that a group of  $b$  input bits can be mapped to one symbol.

This lab considers two of the most common modulations found in commercial wireless systems.

- Binary Phase Shift Keying (BPSK) has

$$\mathcal{C} = \{+1, -1\}.$$

This is arguably the simplest constellation in use. The bit labeling is provided in Table 1. BPSK is a real constellation since it does not contain an imaginary component.

- Quadrature phase shift keying (QPSK) is a complex generalization of BPSK with

$$\mathcal{C} = \{1 + j, -1 + j, -1 - j, 1 - j, \}.$$

Essentially QPSK uses BPSK for the real and BPSK for the imaginary component. QPSK is also known as 4-QAM. The bit labeling is provided in Table 1.

For implementation and analysis, it is convenient to normalize the constellation to have unit energy. This means scaling the constellation such that

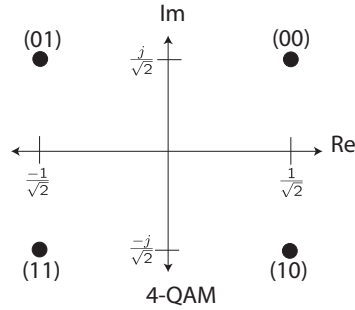
$$\frac{1}{M} \sum_{m=0}^{M-1} |s_m|^2 = 1.$$

The normalized QPSK constellation is illustrated in Figure 2.

The symbols are pulse-shaped and scaled by  $\sqrt{E_x}$  to produce the complex baseband signal

$$x(t) = \sqrt{E_x} \sum_{n=-\infty}^{\infty} s[n]g_{\text{tx}}(t - nT). \quad (1)$$

The scaling factor  $E_x$  is used to model the effect of adding energy or power to  $x(t)$ . The scaling factor will be added typically in the RF by controlling the gain of the transmit power amplifier. The pulse-shape is given by the function  $g_{\text{tx}}(t)$ . To preserve energy, the pulse-shaping function is assumed to be normalized such that  $\int_{-\infty}^{\infty} |g(t)|^2 dt = 1$ . The symbol period is given by  $T$ . The symbol rate is  $1/T$ .



**Figure 2:** QPSK or 4-QAM constellation with generally accepted bit to symbol mappings based on Gray labeling.

| Input Bit | Symbol |
|-----------|--------|
| 0         | 1      |
| 1         | -1     |

(a)

| Input Bits | Symbol   |
|------------|----------|
| 00         | $1 + j$  |
| 10         | $-1 + j$ |
| 11         | $-1 - j$ |
| 01         | $1 - j$  |

(b)

**Table 1:** Symbol Constellation Mappings for (a) BPSK and (b) QPSK

We refer to  $x(t)$  as a complex pulse amplitude signal because complex symbols modulate a succession of pulses given by the function  $g_{tx}(t)$ . Effectively, symbol  $s[n]$  rides the pulse  $g_{tx}(t - nT)$ . The **symbol rate** corresponding to  $x(t)$  in Eq. (1) is  $1/T$ . The units are symbols per second. Typically in wireless systems this is measured in kilo-symbols per second or mega-symbols per second. Be careful to note that  $1/T$  is not necessarily the bandwidth of  $x(t)$ , which depends on  $g_{tx}(t)$ . The **bit rate** is  $b/T$  (where recall there are  $2^b$  symbols in the constellation) and is measured in bits per second. The bit rate is a measure of the number of bits per second the baseband waveform  $x(t)$  carries.

Complex baseband signals are used for notational convenience. The signal  $x(t)$  is complex because the resulting wireless signal is upconverted to “ride” on a carrier frequency. After upconversion, the resulting passband

signal will be  $x_p(t) = \text{Re}\{x(t)\} \cos(2\pi f_c t) - \text{Im}\{x(t)\} \sin(2\pi f_c t)$ . Up-conversion is performed in the analog hardware. The signal  $x_p(t)$  will be launched from the transmit antennas into the propagation environment.

For practical implementation, the pulse-shaping is performed in discrete-time. One way to implement the pulse-shaping is illustrated in Figure 1. The idea is to use a discrete-to-continuous converter operating with a sample period of  $T_x = T/L$  where  $L > 0$  is an integer that denotes the oversampling factor. The symbol sequence is upsampled by  $L$  and then filtered by the oversampled pulse-shaping filter defined by  $g_{tx}[n] := g_{tx}(nT_x)$ .

## 1.2 Detection at the Receiver

When a signal propagates through the wireless channel, it will be affected by noise and other types of channel distortion. The type of distortion impacts the architecture of the receiver and the design of the signal processing algorithms. In this lab, it is assumed that the transmitted signal is only impaired by AWGN. This assumption will gradually be relaxed in future labs.

The AWGN communication channel is a good model for the impairments due to thermal noise present in any wireless communication system. Mathematically

$$z(t) = x(t) + v(t),$$

where  $x(t)$  is the complex baseband signal,  $v(t)$  is the AWGN, and  $z(t)$  is the observation. The assumption that  $v(t)$  is AWGN means the noise is an independent and identically distributed (i.i.d.) complex Gaussian random variable. In the presence of thermal noise, the total variance is  $\sigma_v^2 = N_o = kT_e$ , where  $k$  is Boltzman’s constant  $k = 1.38 \times 10^{-23} \text{ J/K}$  and the effective noise temperature of the device is  $T_e$  in Kelvins. The effective noise temperature is a function of the ambient temperature, the type of antennas, as well as the material properties of the analog front end.

Under the assumed transmitted structure, the optimum receiver structure involves matched filtering, sampling at the symbol rate, and detection. The structure that will be implemented in this lab is illustrated in Figure 1. The received signal  $z(t)$ , which is assumed to be bandlimited, is sampled with a sampling rate of  $1/T_z$ . The sampling rate should be such that Nyquist is satisfied. In this lab we assume that  $T_z = T/M$  where  $M$  is the integer oversampling factor. The received signal is filtered by  $g_{rx}[n] = g_{rx}(nT_z)$  where  $g_{rx}(t) = g_{tx}(-t)$  is the matched filter. After the digital filtering, the signal is downsampled by  $M$  to produce the symbol-rate sampled signal given by  $y[n]$ . The digital filtering is performed such that  $y[n]$  is the equivalent to  $y(t) = \int_t g_{rx}(\tau) z(t - \tau) d\tau$  sampled at  $nT$ .

Under assumptions about the transmit and receive pulse-shapes, described in more detail in Part 2 of Lab 2, a model for the received signal is

$$y[n] = \sqrt{E_x}s[n] + v[n], \quad (2)$$

where  $v[n]$  is i.i.d. complex Gaussian noise with  $\mathcal{N}_c(0, \sigma_v^2)$  where  $\sigma_v^2 = N_o$  for complex thermal AWGN. More background on matched filters and pulse-shapes is provided in Part 2 of Lab 2.

A symbol detector is an algorithm that given the observation  $y[n]$  produces the best  $\hat{s}[n] \in \mathcal{C}$  according to some criterion. In this lab we consider the maximum likelihood (ML) criterion which solves

$$\hat{s}[n] = \arg \max_{s \in \mathcal{C}} f_{y|s}(y[n]|s[n] = s), \quad (3)$$

where  $f_{y|s}(\cdot)$  is the conditional distribution of  $y[n]$  given  $s[n]$  and is known as the likelihood function. For additive white Gaussian noise, the conditional probability has a simple form. This allows the detection problem in Eq. (3) to be simplified to

$$\hat{s}[n] = \arg \min_{s \in \mathcal{C}} \ln f_{y|s}(y[n]|s[n] = s) = \arg \min_{s \in \mathcal{C}} |y[n] - \sqrt{E_x}s|^2. \quad (4)$$

Essentially the algorithm works as follows. Given an observation  $y[n]$ , it determines the transmitted symbol  $s \in \mathcal{C}$ , scaled by  $\sqrt{E_x}$ , that is closest to  $y[n]$  in terms of the euclidean distance in the error term  $y[n] - \sqrt{E_x}s$ . The algorithm in Eq. (4) can be simplified by exploiting structure and symmetry in the constellation.

*Note: If  $x$  is a complex symbol, then  $\|x\|^2 = x_{\text{real}}^2 + x_{\text{imag}}^2$ , where  $x_{\text{real}}$  and  $x_{\text{imag}}$  are the real and imaginary parts of  $x$  respectively.*

## 2 Pre-Lab

To prepare for this part of the lab you will be required to build the modulation and decoding blocks of Figure 1. Tables 2 and 3 describe the details of the VIs you must implement. Plug your code into transmitter.vi and receiver.vi and run Simulator.vi (see the VI hierarchy in Figure 3) to verify functionality and correctness. Do not use the Modulation Toolkit (MT) VIs to implement these two VIs (i.e., do not just create a wrapper around an MT VI).

*Note: From this point forward, please uniquely name your VIs so that they do not exactly match those of the VIs in the simulator (i.e., instead of modulate.vi, use yourname\_modulate.vi).*

Note that both *modulate.vi* and *decode.vi* do not have an input to specify modulation type. Modulation type is a parameter that can be found bundled in the Modulation Parameters cluster, which has already been inserted into the template VIs for you.

**Table 2:** Description of *modulate.vi*

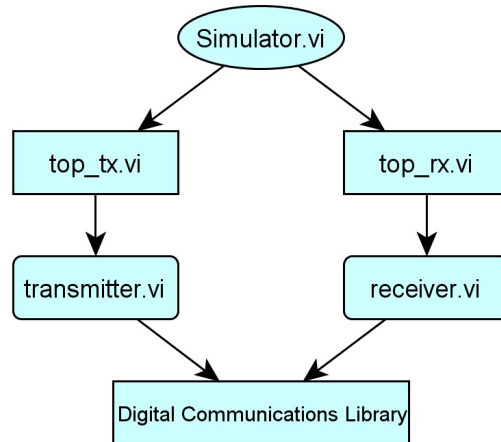
| <i>modulate.vi</i> - maps bit sequence to symbols using appropriate constellation; modifies symbol energy of output symbol sequence |                           |                                |                                                                       |
|-------------------------------------------------------------------------------------------------------------------------------------|---------------------------|--------------------------------|-----------------------------------------------------------------------|
| <b>INPUTS</b>                                                                                                                       | <i>input bit sequence</i> | array of U8 (unsigned bytes)   | input bit sequence (each array element takes on value 0 or 1)         |
|                                                                                                                                     | <i>modulation type</i>    | string                         | determines type of constellation map (i.e., BPSK or QPSK)             |
|                                                                                                                                     | <i>Symbol Energy</i>      | double                         | determines the per-symbol energy of the output sequence (default = 1) |
| <b>OUTPUTS</b>                                                                                                                      | <i>output symbols</i>     | array of CDB (complex doubles) | appropriately modulated output symbols                                |

**Table 3:** Description of *decode.vi*

| <i>decode.vi</i> - performs ML detection on a given symbol sequence |                               |                                |                             |
|---------------------------------------------------------------------|-------------------------------|--------------------------------|-----------------------------|
| <b>INPUTS</b>                                                       | <i>input symbols</i>          | array of CDB (complex doubles) | input symbols to be decoded |
| <b>OUTPUTS</b>                                                      | <i>estimated bit sequence</i> | array of U8 (unsigned bytes)   | decoded output bit sequence |

You have been provided with templates for the VIs you need to create for this lab that already have all the inputs and outputs wired for you. What is required of you is to finish constructing the block diagram to provide the functionality of the VIs.

Throughout the course, several of the VIs you will create will have a *modulation parameters* cluster passed into and out of them. The *modulation parameters in* contains many of the parameters needed by your VIs and will be unbundled from them. This has already been done for you in the template VIs if necessary. Some VIs will also have *modulation parameters out*, so that the cluster can be passed on through to the VIs that follow. Please ensure that these clusters remain wired the way they are in the template VIs, as changing how they are wired will cause VIs further down the line to break.



**Figure 3:** Simulator hierarchy.

Error clusters will allow you to propagate error messages through your code, which will help in debugging your LabVIEW code. If a subVI in your code requires an error cluster as an input, wire *error in* to the appropriate subVI before passing it on to *error out*; otherwise, directly connect these two terminals together.

After you have created your subVIs properly (i.e., wiring all inputs and outputs and editing the icon appropriately), insert them into the simulator provided to you for this lab. To do this, first open up *transmitter.vi* and *receiver.vi* and replace the modulate and decode subVIs with your own (refer to Figures 5 and 6 for where to find these subVIs in the block diagrams). Then you can open up *Simulator.vi* and, using the default parameters, run the simulator to verify that your VIs operate according to the specifications provided in Tables 2 and 3.

Using the simulator and the code you have written, plot average bit-error rate (BER) versus signal-to-noise ratio as in Part 1 of Lab 1, but make sure to obtain data for both BPSK and QPSK modulation<sup>1</sup>. Remember that if signal power is held constant, then decreasing noise power,  $N_0$ , is equivalent to increasing SNR [i.e., you can assume unit power, giving  $\text{SNR} = -N_0$  (in dB)]. Also, to observe errors at high SNR (or low  $N_0$ ) you need to run significantly more iterations in the simulator.

<sup>1</sup>Please ensure that the y-axis is in a logarithmic scale.

Answer the following questions about PSK modulation:

1. PSK is a constant envelope modulation scheme. This means that all symbols have equal energy after modulation. What is the energy of the BPSK and QPSK modulated symbols shown in Tables 1(a) and 1(b) respectively?
2. In *modulate.vi*, what should be the default value for the *Symbol Energy* of the modulated sequence?
3. Consider a uniformly distributed random bit stream  $\{b_n\}$  (i.e.,  $\text{Prob}\{b_n = 1\} = \text{Prob}\{b_n = 0\}$ ). Suppose  $\{b_n\}$  is BPSK modulated such that each symbol has energy  $E_s$ . In an AWGN channel, what is the average probability of bit error as a function of SNR when using ML estimation? You may assume that noise is a real-valued Gaussian random variable with variance  $\frac{N_0}{2}$ , [i.e.,  $\mathcal{N}(0, \frac{N_0}{2})$ ].

*HINT: you may leave the answer in terms of the Q-function, which is derived from the  $\mathcal{N}(0, 1)$  distribution.*

### Pre-Lab Turn In

1. Submit your implementation of *modulate.vi* and *decode.vi*. Remember, you will be penalized if you do not wire the error cluster inputs and outputs in your code. *Note: If you need to generate any additional subVIs for this or any future labs, please submit these subVIs along with your other pre-lab VIs.*
2. Turn in a plot which compares the BER curves for BPSK and QPSK modulation. You will need to collect average BER statistics using your code for SNR values of 0 to 10 dB (in 2 dB increments). Remember to use a logarithmic scale for BER and a dB scale for SNR in your plot<sup>2</sup>. You must use your LabVIEW code to generate the data for your plots.
3. Submit your answers to all of the questions in the pre-lab section.

## 3 Lab Experiment

In this lab you will run your implementation of *modulate.vi* and *decode.vi* over a real wireless link. Remember to use your version of *source.vi* and *error\_detect.vi*, which you have already built in Part 1 of Lab 1. Using your

<sup>2</sup>*HINT: If  $X$  is a real number, then  $X_{dB} = 10 \log X$ . Thus,  $SNR_{dB} = 10 \log(\frac{E_s}{N_0})$ , where  $E_s$  is the average signal power (or symbol energy) and  $N_0$  is average noise power.*

code from Part 1 of Lab 1 in conjunction with the new blocks you have created, you will complete the framework for the transmitter and receiver blocks in lab. This framework will be provided in lab. The task of plugging your code into this framework may seem trivial at this point, but as the course progresses you will incrementally re-implement each block in the framework.

Figure 4 describes the hierarchy of files you will be using in lab. Rounded rectangles in the block diagram represent files which you will be able to alter. The files *top\_tx.vi* and *top\_rx.vi* are the top level of the transmitter and receiver respectively. This level connects the digital communications blocks of the transmitter and receiver (in *transmitter.vi* and *receiver.vi*) with the VIs needed to control the NI RF hardware. You explored the RF hardware and these control VIs in Lab 1, and will need to recall how to modify the parameters/settings for the RF hardware using these VIs.

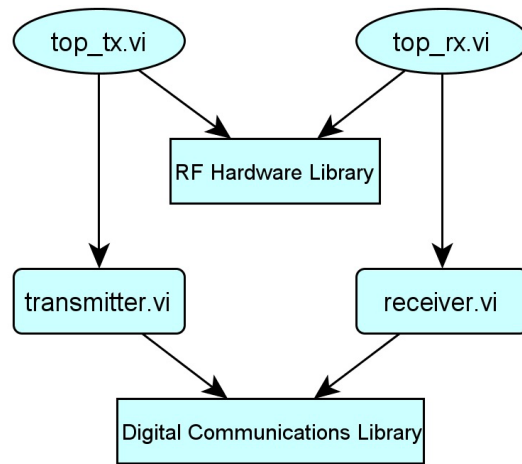


Figure 4: Hierarchy of code framework used in lab.

### 3.1 Insert Your Code

The block diagram of *transmitter.vi* is shown in Figure 5. Replace *source.vi* and *modulate.vi* with your code by right-clicking on the appropriate VI and selecting **Replace ⇒ Select a VI**.

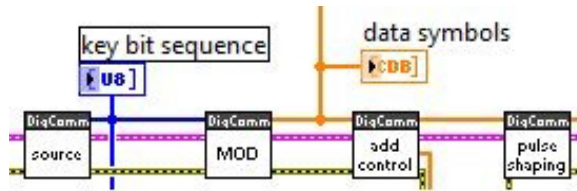


Figure 5: Block diagram of *transmitter.vi*.

Similarly, you will need to replace *decode.vi* and *error.detect.vi* in *receiver.vi* with your code. The block diagram of *receiver.vi* is shown in Figure 6. Add error cluster appropriately.

### 3.2 Communication Over a Wireless Link

Before you begin testing your code over a real wireless link, set the following baseband parameters on the front panel of *top.tx.vi* and *top.rx.vi*:

- Packet length = 100 bits
- Modulation type = QPSK

Leave all other modulation parameters set to their default value. The following RF parameters have already been set up in the HW parameters tab of *top.tx.vi* and *top.rx.vi* for you:

- Carrier frequency
- TX Gain
- RX Gain
- Trigger Level
- Capture Time

The wireless digital communication system used in lab generates a small burst of data, transmits it over a wireless link, and decodes the received signal. To run the system:

1. Start the top level of your transmitter (i.e., run *top.tx.vi*).
2. Wait until the *Transmitting* indicator is lit.
3. Start the top level of your receiver (i.e., run *top.rx.vi*).

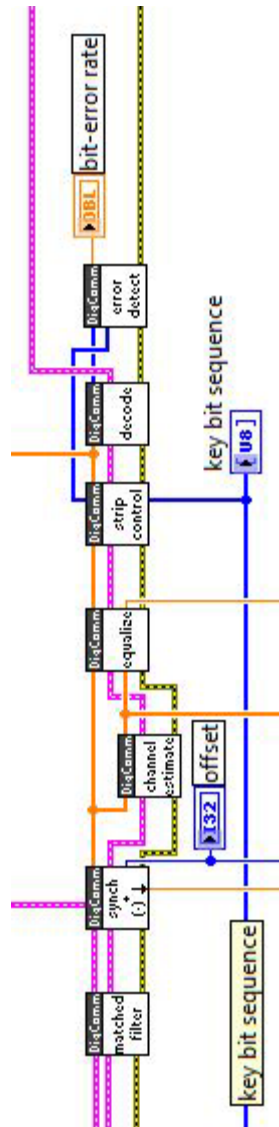


Figure 6: Block diagram of receiver.

Open the front panel of *top\_rx.vi* to see the constellation of the received signal. Note that a queue has been used to pass the key bit sequence to the receiver for error detection. Also notice, on the front panel of *top\_tx.vi* there is a control for the channel model parameters used by *transmitter.vi*. These parameters are used to apply a desired channel model to the transmitted signal. The *real* wireless channel will convolve with this artificially imposed channel model to create a kind of equivalent channel.

Answer the following questions about the digital communication system in lab:

1. What is the name of the queue used to pass the generated bit sequence from *top\_tx.vi* to *top\_rx.vi* (i.e., the bit sequence needed to perform error detection)?
2. Describe what happens to the received constellation as you increase noise power in the top level of the transmitter, (i.e., *top\_tx.vi*).
3. Based on your observation of the received signal constellation, explain which modulation scheme (BPSK or QPSK) performs better (on average) in noisy channels, and why. Assume that both schemes are using the same average symbol energy.

### Lab Turn In

Demonstrate to the instructor that your code is working properly. You will need to show the instructor that both BPSK and QPSK modulation work in your system. Please answer the questions above. You will be required to submit these answers in a future lab report (i.e., in Lab 2, Part 2).